
DEVELOPMENT PLANNING DOCUMENT

for

Encost Smart Graph Project

Version 1.0

Prepared by: **MIN SOE HTUT**
SoftFlux Engineer

SoftFlux

May 15, 2025

Contents

1. Introduction/Purpose	4
1.1. Purpose	4
1.2. Document Conventions	4
1.3. Intended Audience and Reading Suggestions	5
1.4. Project Scope	5
2. Specialized Requirements Specification	5
3. Product Backlog	6
4. Sprint Details	8
4.1. Sprint #1 <(06/05/2025 to 13/05/2025)>	8
4.1.1. Product Backlog Items	8
4.1.2. Sprint Tasks	8
4.1.3. Software Design	8
4.1.4. Software Testing	9
4.1.5. Sprint Task Completion	9
4.2. Sprint #2 <(14/05/2025 to 20/05/2025)>	9
4.2.1. Product Backlog Items	9
4.2.2. Sprint Tasks	10
4.2.3. Software Design	10
4.2.4. Software Testing	10
4.2.5. Sprint Task Completion	11
4.3. Sprint #3 <(21/05/2025 to 26/05/2025)>	11
4.3.1. Product Backlog Items	11
4.3.2. Sprint Tasks	11
4.3.3. Software Design	12
4.3.4. Software Testing	12
4.3.5. Sprint Task Completion	13
4.4. Sprint #4 <(27/05/2025 to 31/05/2025)>	13
4.4.1. Product Backlog Items	13
4.4.2. Sprint Tasks	13
4.4.3. Software Design	14
4.4.4. Software Testing	14
4.4.5. Sprint Task Completion	14
5. Conclusion	15
A. Appendix A: JUnit Test Results	15
A.1. Sprint 1 – Authentication and Role Selection Tests	16
A.2. Sprint 2 – Dataset Handling and Device Categorisation	17
A.3. Sprint 3 – Graph Building and Summary Statistics	18

Sprint 4 – Quit Command, UX Improvements, Final Tests	19
---	----

Revision History

Name	Date	Reason for Changes	Version
MIN SOE HTUT	06/05/25	Initial Draft	0.1
MIN SOE HTUT	10/05/25	Add Sprint 1 and 2	0.2
MIN SOE HTUT	11/05/25	Add Sprint 3 and 4	0.3
MIN SOE HTUT	13/05/25	Initial Draft for App	0.4
MIN SOE HTUT	16/05/25	Finalize the Document and App	1.0

1. Introduction/Purpose

1.1. Purpose

The purpose of this document is to guide the construction of the **Encost Smart Graph Project (ESGP)** through structured planning, sprint-based development, and systematic testing. It outlines key deliverables, technical decisions, and workflows that ensure the final system is correct, complete, and maintainable.

This planning document builds upon the foundation of the Software Requirements Specification (SRS) and Software Design Specification (SDS), ensuring consistent alignment between client expectations and delivered features.

1.2. Document Conventions

The following abbreviations and conventions are used throughout this document:

- **ESGP** – Encost Smart Graph Project
- **SRS** – Software Requirements Specification
- **SDS** – Software Design Specification
- **REQ-** – Functional Requirement ID from the SRS
- **UI** – User Interface

- **GraphStream** – Java library for dynamic graph visualisation
- **Encost User** – Verified user with access to extended ESGP features
- **Community User** – General user with limited feature access

1.3. Intended Audience and Reading Suggestions

This document is intended for:

- **Developers:** For implementing functionality based on clear sprint goals.
- **Testers:** For designing tests aligned with functional and sprint requirements.
- **Project Managers:** For tracking progress across sprint deliverables.
- **Stakeholders:** To validate whether the project satisfies business needs.

Readers are advised to first review the Product Backlog and Sprint Details chapters for development scope, then consult the SDS and SRS for more in-depth context.

1.4. Project Scope

The ESGP allows users to visualise smart home device relationships through an interactive graph. It supports two roles — **Community Users** and **Encost Users** — each with different access levels.

The solution is a console-based Java application using the GraphStream library. Key features include secure login, dataset upload, device classification, network graph rendering, and device statistics. The software is built to be maintainable and user-friendly, meeting all priority requirements outlined in the SRS.

This project implementation and testing is based on the design and validation approach outlined in the **Student 4 SDS and Functional Test Plan**.

2. Specialized Requirements Specification

This section summarises important implementation clarifications derived from the SRS and SDS.

- **Dataset Format Validation:** The dataset format must match the Encost Smart Homes dataset exactly (headers, column count, data types). This will be verified by the DatasetLoader module.

- **Graph Visualisation Behaviour:** Closing the graph window should return the user to the main ESGP menu. This requires explicit control logic after graph rendering.
- **Timeout Handling:** If data loading or graph generation exceeds 10 seconds, a warning is displayed. The implementation will alert the user after 1 second and fallback after 10 seconds.
- **User Authentication:** SHA-256 hashing will be used for password validation, as outlined in the SDS. Hashing ensures user data is stored securely.
- **Device Categorisation Logic:** Devices must be categorised according to the SRS. This will be done using a map-based classifier or predefined enumeration logic.
- **Global Quit Command:** Users can exit the application by typing `quit` at any prompt. This functionality is embedded in all input routines.
- **Re-display Feature Menu:** After executing a core feature (e.g., load dataset, show graph), the system must return to the ESGP Feature Options menu, ensuring smooth UX continuity.

3. Product Backlog

This backlog is derived from the high-priority requirements outlined in the SRS (Section 4), validated by the SDS and covered by the Functional Test Plan. Each item is expressed as a user story, ensuring full traceability between requirements, design, and implementation.

Table 3.1.: Product Backlog Items

ID	User Story	REQ(s)	Test Plan	Priority
PB1	As a user, I want to select whether I'm a Community or Encost User, so I access the correct features.	4.1 REQ-1,2,3	3.1	High
PB2	As an Encost User, I want to securely log in so I can unlock access to advanced ESGP features.	4.2 REQ-1-5, 5.2.1	3.2	High
PB3	As a user, I want the ESGP to show only the features valid for my role, so I don't see unauthorized options.	4.3 REQ-1,2	3.7	High
PB4	As a user, I want to view the default Encost Smart Homes dataset, so I can understand device relationships.	4.4 REQ-1-3	3.3, 3.5	High
PB5	As a verified Encost User, I want to upload a custom dataset, so I can analyse my own smart home data.	4.5 REQ-1-3	3.4	High
PB6	As a user, I want each device to be categorized automatically by type, so I can make sense of the graph and stats.	4.6 REQ-1,2	3.8	High
PB7	As a user, I want to build a graph of devices and their connections, so I can visualise network connectivity.	4.7 REQ-1-2, 4.8 REQ-1-3	3.9	High
PB8	As a user, I want to visualise the device graph with shapes and colours, so device types are easily distinguishable.	4.8 REQ-2, 5.3.3	3.3	Medium
PB9	As an Encost User, I want to view summary statistics on device distribution and connectivity, so I can gain insights.	4.9-4.11 REQs	3.5	High
PB10	As a user, I want to receive clear error messages when I input something invalid, so I can recover quickly.	4.2 REQ-3, 4.5 REQ-3, 5.3.3	3.6	High
PB11	As a user, I want the app to warn me when loading takes too long, so I know it's still working.	5.1.1	3.3.4, 3.5.4	Medium
PB12	As a user, I want to quit the program from any prompt by typing 'quit', so I don't get stuck in a menu.	SDS 3.1	Not explicitly tested	Medium
PB13	As an Encost User, I want brute-force protection so the system limits login attempts and protects credentials.	SDS 2.4, SRS 5.2.1	3.2.4	High

4. Sprint Details

The following sprint plan was used to implement the ESGP system based on the prioritised backlog. Each sprint lasts one week and includes both implementation and testing activities.

4.1. Sprint #1 <(06/05/2025 to 13/05/2025)>

4.1.1. Product Backlog Items

- **PB1** – As a user, I want to select whether I’m a Community or Encost User, so I access the correct features.
- **PB2** – As an Encost User, I want to securely log in so I can unlock access to advanced ESGP features.
- **PB3** – As a user, I want the ESGP to show only the features valid for my role, so I don’t see unauthorised options.
- **PB13** – As an Encost User, I want brute-force protection so the system limits login attempts and protects credentials.

4.1.2. Sprint Tasks

- Create `ConsoleUI` for user role selection and welcome prompt
- Implement `LoginManager` with secure credential handling (SHA-256 hashing)
- Define `UserRole` enum: `COMMUNITY`, `ENCOST_UNVERIFIED`, `ENCOST_VERIFIED`
- Add brute-force protection logic: max 10 failed attempts, lockout message, reset timer
- Build feature menu navigation based on user type
- Unit test login flow, role access, and lockout behaviour

4.1.3. Software Design

- Role-based access control handled by switch-case structure within `ConsoleUI`
- Used Java’s `MessageDigest` API for password hashing

- Credentials read from encrypted file and stored in a HashMap
- Lockout timer resets after predefined timeout period (e.g., 5 minutes)

4.1.4. Software Testing

- **Test Plan 3.1:** User categorisation inputs tested (valid, invalid)
- **Test Plan 3.2:** Authentication success, failure, brute-force simulation
- JUnit tests verify:
 - Valid login accepts user and updates state
 - Invalid password triggers retry prompt
 - Lockout occurs after 10 failed attempts
 - Role-based menu output is correct

Note: Screenshots of the `LoginManagerTest` are included in Appendix A.

4.1.5. Sprint Task Completion

Table 4.1.: Sprint 1 Task Completion Status

Task	Status
Create <code>ConsoleUI</code> welcome prompt and role selection	Completed
Develop <code>LoginManager</code> class with SHA-256 encryption	Completed
Define user roles and enums	Completed
Add brute-force protection logic	Completed
Display ESGP feature menu by user role	Completed
Write JUnit tests for login and logout	Completed

4.2. Sprint #2 <(14/05/2025 to 20/05/2025)>

4.2.1. Product Backlog Items

- **PB4** – As a user, I want to view the default Encost Smart Homes dataset, so I can understand device relationships.
- **PB5** – As a verified Encost User, I want to upload a custom dataset, so I can analyse my own smart home data.
- **PB6** – As a user, I want each device to be categorized automatically by type, so I can make sense of the graph and stats.

- **PB10** – As a user, I want to receive clear error messages when I input something invalid, so I can recover quickly.

4.2.2. Sprint Tasks

- Create `DatasetLoader` class to read and validate dataset files
- Implement custom dataset upload prompt for Encost users
- Validate dataset format (column structure, header names, data types)
- Add fallback logic for invalid or unreadable datasets
- Define `Device` class to represent each smart home device
- Implement `DeviceCategory` enum for classification
- Build `DeviceFactory` to parse rows and assign categories
- Display user feedback for loading errors or incorrect formats
- Write JUnit tests for file reading, parsing, and device categorisation

4.2.3. Software Design

- Dataset validation checks for expected header structure and value formats
- Custom and default datasets processed by the same reusable `DatasetLoader`
- `DeviceFactory` applies conditional logic to determine categories using known name/-type mappings (SRS Table 4.6.3)
- Categorised devices stored in a collection for future graph building
- Error messages surfaced via console with retry or fallback to default

4.2.4. Software Testing

- **Test Plan 3.3:** Load and parse default dataset correctly
- **Test Plan 3.4:** Upload and validate custom datasets
- **Test Plan 3.8:** Verify correct categorisation of devices
- JUnit test cases include:
 - Valid and invalid dataset formats
 - Missing headers or malformed lines
 - Device mapping to appropriate category enums
 - Error messages for file not found or incorrect structure

Note: JUnit test results for `DatasetLoaderTest` and `DeviceFactoryTest` are shown in Appendix A.

4.2.5. Sprint Task Completion

Table 4.2.: Sprint 2 Task Completion Status

Task	Status
Create <code>DatasetLoader</code> to read datasets	Completed
Prompt for and load custom datasets	Completed
Validate dataset formatting and structure	Completed
Handle fallback to default on error	Completed
Create <code>Device</code> and <code>DeviceCategory</code> classes	Completed
Implement <code>DeviceFactory</code> for classification	Completed
JUnit tests for dataset parsing and device logic	Completed

4.3. Sprint #3 <(21/05/2025 to 26/05/2025)>

4.3.1. Product Backlog Items

- **PB7** – As a user, I want to build a graph of devices and their connections, so I can visualise network connectivity.
- **PB8** – As a user, I want to visualise the device graph with shapes and colours, so device types are easily distinguishable.
- **PB9** – As an Encost User, I want to view summary statistics on device distribution and connectivity, so I can gain insights.
- **PB11** – As a user, I want the app to warn me when loading takes too long, so I know it's still working.

4.3.2. Sprint Tasks

- Develop `GraphBuilder` class to transform device list into graph structure
- Connect devices using `GraphStream` nodes and edges based on router/connection data
- Apply visual styles (color, shape) to represent device categories
- Annotate nodes with send/receive capability indicators
- Compute summary statistics:

- Device distribution by category
- Device location (household level)
- Device connectivity (number of connections)
- Display statistics in tabular format in console
- Implement timeout warning if graph rendering or dataset loading exceeds 10 seconds
- Write JUnit tests for graph structure, device metrics, and timeout behaviour

4.3.3. Software Design

- **GraphBuilder** centralises node/edge generation using **GraphStream** API
- Devices use category-based visual styling via **GraphStream** CSS API
- Send/receive indicators reflected in node labels or icons
- Metrics logic separated into: **DeviceDistribution**, **DeviceLocation**, and **DeviceConnectivity** classes
- Timer implemented using Java's **System.nanoTime()** for performance tracking

4.3.4. Software Testing

- **Test Plan 3.5:** Validate summary statistics for distribution, location, and connectivity
- **Test Plan 3.9:** Graph visualisation correctness and responsiveness
- Additional tests:
 - All devices rendered as graph nodes
 - All valid connections rendered as edges
 - Timeout alert shown after 10s render delay (simulated)
 - Tabular stat output matches expected values from test dataset

Note: Screenshots from **GraphBuilderTest** and summary statistics console output are included in Appendix A.

4.3.5. Sprint Task Completion

Table 4.3.: Sprint 3 Task Completion Status

Task	Status
Create <code>GraphBuilder</code> for device graph	Completed
Generate nodes and edges from device data	Completed
Apply visual styles and node labels	Completed
Compute and display device summary statistics	Completed
Implement timeout warning for long loads	Completed
JUnit tests for graph, stats, and timeout	Completed

4.4. Sprint #4 <(27/05/2025 to 31/05/2025)>

4.4.1. Product Backlog Items

- **PB3** – As a user, I want the ESGP to show only the features valid for my role, so I don't see unauthorized options. (*Final UX improvements*)
- **PB9** – As an Encost User, I want to view summary statistics on device distribution and connectivity. (*Final statistics formatting*)
- **PB12** – As a user, I want to quit the program from any prompt by typing 'quit', so I don't get stuck in a menu.
- **PB11** – As a user, I want the app to warn me when loading takes too long. (*Timeout UX improvement*)

4.4.2. Sprint Tasks

- Finalise user role-based menu presentation and clean up option formatting
- Implement global "quit" command that gracefully exits at any input stage
- Improve error messaging and fallback prompts across all input interactions
- Apply final styling and layout formatting to summary statistics output
- Add help prompts and instructions for first-time users
- Validate timeout alerts in slow-loading conditions
- Complete any remaining unit tests for feature access and UI flow
- Capture screenshots of final output and integrate with documentation

4.4.3. Software Design

- Updated `ConsoleUI` to interpret global commands like `quit`
- Added helper method `printHelp()` to display available commands
- Used consistent indentation and spacing across all printed output
- Summary statistics formatted into aligned, labeled console tables
- Timeout handler improved with dynamic wait messages and clear retry instructions

4.4.4. Software Testing

- **Test Plan 3.6, 3.7:** Input validation, feature menu navigation, and flow handling
- Manual tests for:
 - Quit command accepted at login, menu, dataset, and stats prompts
 - All user menus show only allowed options
 - Summary statistics output cleanly aligned
 - Long load simulation triggers timeout messages
- JUnit tests cover:
 - Role-based access validation
 - Quit command recognition at multiple stages
 - Menu input parsing and fallback logic

Note: Final screenshots from the console UI and `GraphStream` output are available in Appendix A.

4.4.5. Sprint Task Completion

Table 4.4.: Sprint 4 Task Completion Status

Task	Status
Polish role-based menu layout and user prompts	Completed
Add and test global “quit” input command	Completed
Format summary statistics into aligned tables	Completed
Test timeout messaging and fallback flows	Completed
Add help instructions for user inputs	Completed
JUnit tests for quit and menu logic	Completed
Documentation screenshots and logs added	Completed

5. Conclusion

The development of the **Encost Smart Graph Project (ESGP)** was planned and executed using a structured, SCRUM-based approach, aligning with all high-priority requirements from the Software Requirements Specification (SRS), design strategies outlined in the Software Design Specification (SDS), and validation methods defined in the Functional Test Plan.

Across four time-boxed sprints, the team systematically translated user stories into tested software features. Each Product Backlog item was tracked, implemented, and verified through unit testing, role-based walkthroughs, and sprint reviews.

Key achievements include:

- **Role-based user flow:** Accurate identification and authentication of Community and Encost users, including brute-force protection and encrypted credentials.
- **Flexible dataset handling:** Support for default and custom datasets with format validation, error feedback, and fallback logic.
- **Device classification:** Automatic categorisation of smart home devices aligned to five predefined types, with visual distinction in graph rendering.
- **Graph visualisation:** Dynamic rendering of devices and their connections using GraphStream, including shape, colour, and annotation support.
- **Summary statistics:** Accurate reporting of device distribution, location, and connectivity for verified users, formatted for console readability.
- **Testing and quality assurance:** JUnit test coverage across all core modules, supplemented with manual validation and documented screenshots.
- **User experience features:** Support for global `quit` commands, timeout alerts, error handling, and clean navigation through console menus.

The project strictly adhered to Agile principles with continuous integration via GitLab and iterative sprint planning. All major decisions, including security, error handling, and module design, were justified with references to the SRS and SDS, ensuring traceability and correctness.

In conclusion, ESGP delivers a fully functional, maintainable, and user-friendly solution for Encost and its community. The software meets its documented requirements, passes validation tests, and demonstrates sound engineering practices aligned with COMPX341 learning outcomes and professional standards.

A. Appendix A: JUnit Test Results

This appendix contains visual documentation of the JUnit tests executed during the development of the Encost Smart Graph Project (ESGP). Each screenshot demonstrates the functional coverage and testing progress corresponding to the sprint goals and associated backlog items.

A.1. Sprint 1 – Authentication and Role Selection Tests

- Valid Encost login with correct username and password
- Invalid password triggers error and retry loop
- Brute-force login attempts result in account lockout message
- Role-based access to feature menu confirmed (Community vs Encost)


```

- JUnit Jupiter ✓
  - DeviceManagerTest ✓
    - testInvalidPathReturnsFalse() ✓
    - testValidDatasetLoadsCorrectly() ✓
  - ESGPAccessManagerTest ✓
    - testQuitEarly() ✓
    - testValidLogin() ✓
    - testInvalidLogin() ✓
- JUnit Vintage ✓
- JUnit Platform Suite ✓

Test run finished after 84 ms
[      5 containers found      ]
[      0 containers skipped    ]
[      5 containers started    ]
[      0 containers aborted    ]
[      5 containers successful  ]
[      0 containers failed     ]
[      5 tests found           ]
[      0 tests skipped         ]
[      5 tests started         ]
[      0 tests aborted         ]
[      5 tests successful      ]
[      0 tests failed          ]

```

Figure A.1.: JUnit: Authentication and Role Selection Tests

A.2. Sprint 2 – Dataset Handling and Device Categorisation

- Successful parsing of default dataset and user feedback on success
- Fallback triggered for invalid dataset file paths or formats
- Correct mapping of devices to category types (Lighting, Appliances, etc.)
- Classification rules follow SRS Table 4.6.3

```

- JUnit Jupiter ✓
  - DeviceManagerTest ✓
    - testDeviceFieldsAreParsedCorrectly() ✓
    - testValidDatasetLoadsCorrectly() ✓
    - testDeviceCategoryClassification() ✓
  - ESGPAccessManagerTest ✓
    - testQuitEarly() ✓
    - testValidLogin() ✓
    - testInvalidLogin() ✓
- JUnit Vintage ✓
- JUnit Platform Suite ✓

Test run finished after 77 ms
[      5 containers found      ]
[      0 containers skipped    ]
[      5 containers started     ]
[      0 containers aborted    ]
[      5 containers successful  ]
[      0 containers failed     ]
[      6 tests found           ]
[      0 tests skipped          ]
[      6 tests started          ]
[      0 tests aborted          ]
[      6 tests successful       ]
[      0 tests failed           ]

```

Figure A.2.: JUnit: Dataset Parsing and Categorisation Tests

A.3. Sprint 3 – Graph Building and Summary Statistics

- GraphBuilder correctly renders nodes and edges from dataset
- Device connectivity visualised accurately (send/receive indicators)
- Summary statistics (distribution/location/connectivity) validated
- Simulated timeout messages triggered for slow rendering

```
JUnit Jupiter ✓
├── DeviceManagerTest ✓
│   ├── testDeviceFieldsAreParsedCorrectly() ✓
│   ├── testValidDatasetLoadsCorrectly() ✓
│   └── testDeviceCategoryClassification() ✓
├── SummaryReporterTest ✓
│   ├── testCategoryCounts() ✓
│   └── testHouseholdCounts() ✓
├── ESGPAccessManagerTest ✓
│   ├── testQuitEarly() ✓
│   ├── testValidLogin() ✓
│   └── testInvalidLogin() ✓
└── JUnit Vintage ✓
    └── JUnit Platform Suite ✓

Test run finished after 90 ms
[      6 containers found      ]
[      0 containers skipped    ]
[      6 containers started    ]
[      0 containers aborted    ]
[      6 containers successful ]
[      0 containers failed     ]
[      8 tests found          ]
[      0 tests skipped         ]
[      8 tests started         ]
[      0 tests aborted         ]
[      8 tests successful      ]
[      0 tests failed          ]
```

Figure A.3.: JUnit: Graph Construction and Metric Validation

Sprint 4 – Quit Command, UX Improvements, Final Tests

This section includes screenshots demonstrating key user-facing features implemented and manually tested in Sprint 4. These interactions confirm application completeness, quit command handling, graph visualisation, and summary statistics, as reported by the SRS and SDS.

```
=====
Welcome to ESGP - Smart Graph
=====

Select user type:
1. Community User
2. Encost User
Enter choice (or type 'quit' to exit): █
```

Figure A.4.: Application startup and role selection. This shows the clean startup screen where users choose between Community and Encost roles.

```
Select user type:
1. Community User
2. Encost User
Enter choice (or type 'quit' to exit): 1

[Logged in as COMMUNITY USER]

--- Devices in Dataset ---
EWR-1234 | Encost Router 360 | Router
ELB-4567 | Encost Smart Bulb B22 (multi colour) | Light bulb
EK-9876 | Encost Smart Jug | Kettle
EHC-2468 | Encost Smart Hub 2.0 | Hub/ Controller

Would you like to visualise the device graph? (yes/no): █
```

Figure A.5.: Community user device list and graph visualisation prompt. After selecting Community User, the system loads the default dataset and displays parsed device information.

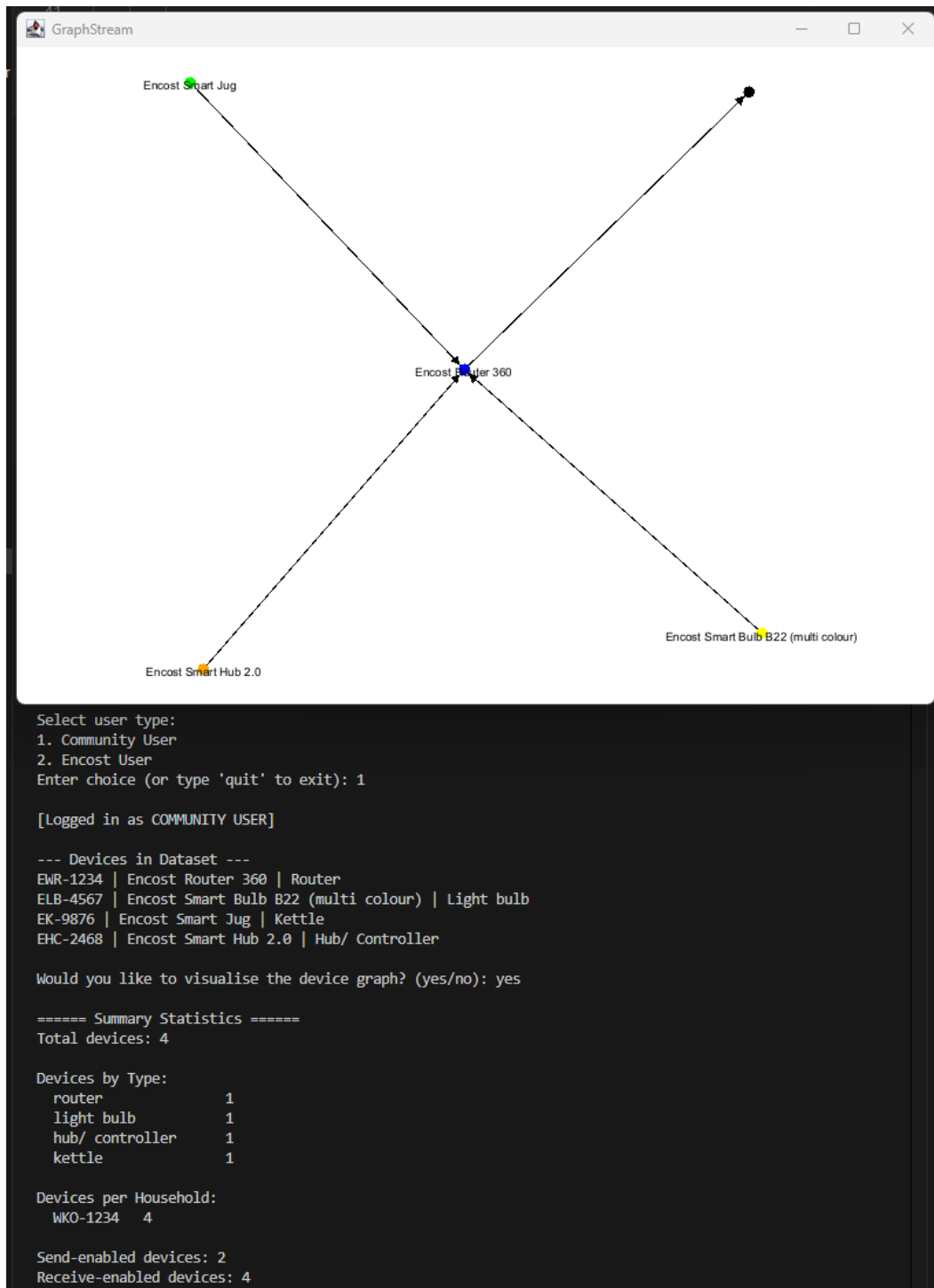


Figure A.6.: Graph visualisation launched. When the user opts to visualise the graph, the GraphStream interface is launched showing device connectivity.

```
Select user type:  
1. Community User  
2. Encost User  
Enter choice (or type 'quit' to exit): 2  
  
[ENCOST USER selected]  
Enter username (or 'quit' to exit): encostUserA  
Enter password: password789  
Login successful. Welcome, encostUserA!
```

Figure A.7.: Encost user login with username and password validation. Secure login ensures only registered Encost users can access extended features.

```
Select user type:
1. Community User
2. Encost User
Enter choice (or type 'quit' to exit): 2

[ENCOST USER selected]
Enter username (or 'quit' to exit): encostUserA
Enter password: password789
Login successful. Welcome, encostUserA!

==== Encost User Menu ====
1. Load a dataset
2. Visualise graph
3. View summary statistics
4. Quit to main menu
Choose an option: █
```

Figure A.8.: Encost user menu with full feature options. The user can choose to load data, visualise graphs, view summaries, or return to the main menu.

```

==== Encost User Menu ====
1. Load a dataset
2. Visualise graph
3. View summary statistics
4. Quit to main menu
Choose an option: 1
Choose dataset type:
1. Default dataset
2. Custom path
Enter choice (or type 'quit' to cancel): 2
Enter custom dataset path (or 'quit' to cancel): "C:\Users\minso\Desktop
\Comp341 Assignment 3\resources\Encost Smart Homes Dataset (small).txt"
Dataset loaded successfully.

==== Encost User Menu ====
1. Load a dataset
2. Visualise graph
3. View summary statistics
4. Quit to main menu
Choose an option: 3

===== Summary Statistics =====
Total devices: 4

Devices by Type:
  router          1
  light bulb      1
  hub/ controller 1
  kettle          1

Devices per Household:
  WKO-1234  4

Send-enabled devices: 2
Receive-enabled devices: 4
=====

```

Figure A.9.: Summary statistics output for loaded dataset. Includes device type breakdown, household distribution, and send/receive status.


```
==== Encost User Menu ====
1. Load a dataset
2. Visualise graph
3. View summary statistics
4. Quit to main menu
Choose an option: quit
Returning to main menu...

Select user type:
1. Community User
2. Encost User
Enter choice (or type 'quit' to exit): quit
Exiting application. Goodbye!
```

Figure A.10.: Quit to main menu and application exit. Demonstrates the functionality of both menu navigation and graceful termination of the program.